

S26 — XML-Based Museum Management Platform

Data Pipeline 1 Project Report & Technical Architecture Specification

1. Group Workload Distribution

The workload was distributed systematically among group members based on technical specialization, ensuring balanced contributions across data modeling, XSLT stylesheet implementation, Python pipeline engineering, and validation testing.

Group Member	Contribution	Precise Tasks Completed
Student A (Lead Architect)	34%	XML Schema (XSD) design, key/keyref relational constraints, historical provenance modeling, Python pipeline validation engine.
Student B (XSLT & UI Lead)	33%	HTML visualization stylesheets (01–06), recursive XSL template matching, CSS styling, conservation aggregate cost calculations.
Student C (Data Integration)	33%	XML/JSON/YAML export stylesheets (07–10), JSON Schema draft-2020-12 alignment, Python DOM API implementation (dom_export.py), report generation.

2. Working Environment & Tooling

The project was developed in a modern software engineering environment utilizing standard XML processing tools:

- **Operating Environment:** Windows 11 64-bit, VS Code IDE with XML Tools extension.
- **Execution Runtime:** Python 3.13 64-bit environment.
- **XML Processing Libraries:** lxml 6.1.2 (for W3C XML Schema validation & XSLT 1.0 engine) and standard Python xml.dom.minidom (for W3C DOM Level 2 tree traversal).
- **Schema Validation:** jsonschema 4.26.0 for JSON Schema Draft 2020-12 validation.
- **Documentation & Reporting:** ReportLab 5.0.1 for programmatic PDF report compilation.

3. Data Modeling Principles & Architecture Discussion

Designing an enterprise XML repository for cultural heritage artifacts requires balancing normalized relational integrity against hierarchical document structure. Rather than embedding nested entities directly inside each artifact, our model uses a **relational ID-based architecture** inside XML.

Modeling Principles & Trade-offs:

- **Relational Integrity vs. Hierarchical Duplication:** Entities such as artists, collections, curators, historicalPeriods, and culturalSites are defined as top-level entity tables under root <m:museum>. Artifacts store compact key references (e.g. artistRef="ART001"). This avoids massive data redundancy, update anomalies, and inconsistencies.
- **Strict Type Constraints & Patterns:** The schema (museum.xsd) enforces fine-grained data types. Unique identifiers use regex pattern restrictions ([A-Z]{1,4}[0-9]{3}). Monetary values use custom complex types (MoneyType) combining decimal precision with ISO currency code attributes.

- **Referential Constraints:** xs:key and xs:keyref constraints enforce primary key uniqueness and foreign key referential integrity at schema validation time.

Specific Modeling Problem & Solution — Artifact Provenance:

Problem: Cultural artifacts possess complex historical provenance records (ownership transfers, commissions, past exhibitions, acquisitions) occurring over centuries. Modeling provenance as simple text strings loses structure, while creating separate relational tables fragments event sequences.

Solution: We designed a modular ProvenanceType containing a sequence of ProvenanceEventType elements embedded directly within ArtifactType (minOccurs=0). Each event records a date, event type (Commission, Transfer, Exhibition, Acquisition), location, and detailed description. This preserves historical chronology while keeping event data coupled with the artifact.

4. Scenarios & XSLT Transformation Architecture

The platform implements 10 distinct XSLT transformation scenarios spanning HTML visualization, XML interchange, JSON API payload generation, and YAML summary export.

ID	Format	Scenario Name & Description
01	HTML	Artifact Catalogue: Complete web catalogue displaying dimensions, locations, status badges, and acquisition histories.
02	HTML	Exhibition Calendar: Chronological exhibition schedule with curator dereferencing and featured work counts.
03	HTML	Artist Explorer: Artist directory displaying biographies, lifespans, and dynamic artifact counts.
04	HTML	Historical Browser: Grouping artifacts by historical period (Renaissance, Baroque, Impressionism).
05	HTML	Loan Dashboard: Management dashboard for international loans (Active, Scheduled, Completed) with insurance values.
06	HTML	Conservation Report: Analytical restoration report computing total/average costs and status statistics.
07	XML	Exhibition Export: Simplified XML interchange format for partner museum portals.
08	XML	Provenance Export: Structuring artifact historical ownership timelines into XML register format.
09	JSON	Artifact API: REST API payload JSON format validated against JSON Schema Draft 2020-12.
10	YAML	Collection Summary: YAML configuration export summarizing holdings by collection ID.

Deep Dive into Complex Scenario Solution — Scenario 06 (Conservation Report):

Scenario 06 represents the most computationally complex XSLT transformation. It performs dynamic mathematical aggregation across conservation records using pure XSLT 1.0 XPath expressions:

1. **Status Aggregation:** Computes live project counts using predicates: count(m:conservationRecord[m:status='completed']).
2. **Financial Accumulation:** Calculates total restoration expenditure using XPath summation: sum(m:conservationRecord/m:cost).
3. **Conditional Division & Formatting:** Avoids division-by-zero errors using <xsl:choose> and formats currency values to 2 decimal places using format-number(sum(...) div count(...), '#,##0.00').
4. **Recursive Template Dereferencing:** Matches each record via <xsl:apply-templates> and looks up artifact titles via key('artifact-by-id', m:artifactRef)/m:title.

5. Dual Python Implementation Architecture

To fulfill the project prompt requirements and demonstrate comprehensive mastery of XML programming APIs, the solution provides **two independent Python processing implementations**:

1. **Primary Pipeline (src/pipeline.py — lxml API):** Parses museum.xml, validates against museum.xsd schema using lxml's C-based XMLSchema engine, discovers all 10 XSLT files under xslt/, applies transformations via etree.XSLT, and outputs files into outputs/.
2. **Second Implementation (src/dom_export.py — W3C DOM API):** Utilizes Python's standard xml.dom.minidom module to parse the XML DOM tree directly. It navigates element nodes, extracts text nodes, performs manual in-memory relational joins across entity lookup dictionaries, and generates a structured JSON summary payload. This satisfies the optional bonus second implementation criteria.

6. Evaluation & Verification Results

The system was validated through automated test suites:

- **XML Schema Validation:** museum.xml validated against museum.xsd with 0 errors.
- **XSLT Transformation Suite:** All 10 XSLT transformations executed with 100% success rate, generating valid HTML, XML, JSON, and YAML outputs.
- **JSON Schema Test:** outputs/json/09_artifact_api.json validated against json-schema/artifact-api.schema.json using jsonschema.Draft202012Validator with 0 validation errors.
- **DOM API Verification:** src/dom_export.py traversed all 8 artifact nodes and 16 provenance events cleanly.

7. Dedicated Section on AI Assistance (Co-Pilot Appendix)

In accordance with project guidelines, this section documents the utilization of AI assistants (Antigravity AI / Gemini 3.6) during project development.

A. Precise Prompts Used:

1. *"Analyze project files and tell me everything and what is missing based on the assignment PDF."*
2. *"Refactor XSLT stylesheets to use idiomatic recursive template matching (<xsl:apply-templates>) instead of procedural for-each loops."*
3. *"Align JSON Schema draft-2020-12 with Scenario 09 JSON output format and implement Python DOM API script."*

B. Initial AI Outputs Received:

- The AI identified several missing requirements: empty report directory, schema mismatches in 4 XSLT files (e.g. 04_historical_browser.xsl searching for m:historicalPeriod instead of m:period; 05_loan_dashboard.xsl searching for @status attributes instead of child elements; 06_conservation_report.xsl searching for m:restorationProject instead of m:conservationRecord).
- The AI generated draft XSLT stylesheets, Python lxml pipeline scripts, and ReportLab PDF compilation code.

C. Human Corrections & Quality Control Applied:

1. **Console Character Encoding Fix:** The AI initial Python script included unicode checkmark characters (  2713), which crashed Windows PowerShell under default cp1252 encoding. We corrected print outputs to ASCII [OK] indicators.
2. **Provenance Schema Alignment:** The AI noted that Scenario 08 (Provenance Export) outputted empty tags because `` was missing in `museum.xsd`. We added `ProvenanceType` to `museum.xsd` and inserted structured event data into `museum.xml`.
3. **XSLT Key Dereferencing:** Corrected Scenario 02 to resolve curator IDs (`CUR001`) into full curator names (`Claire Martin`) via `key('curator-by-id')`.
4. **JSON Schema Alignment:** Corrected JSON Schema property types to match Scenario 09 integer years and nested artist objects.